

FIPU Raspodijeljeni sustavi: Završni kolokvij

VAŽNA NAPOMENA: Predajte samo jednu `zip` datoteku koja sadrži sve potrebne direktorije/datoteke.

- poveznica za predaju nalazi se na Merlinu i Google Chatu kolegija

Zadatak 1 (14 bodova)

1.1 Napravite novi `FastAPI` mikroservis `UserProfileAPI` koji će omogućiti korisnicima da dohвате podatke o korisnicima i dodaju nove korisnike u *in-memory* kolekcije.

1.2 Implementirajte Pydantic model `UserProfile` koji sadrži sljedeće attribute:

- `id` - jedinstveni identifikator korisnika
- `username` - korisničko ime
- `email` - e-mail adresa korisnika
- `full_name` - ime i prezime korisnika
- `bio` - biografija korisnika

1.3 Nadogradite Pydantic model `UserProfile` na način da:

- dodate atribut `friends` koji će se validirati kao **lista korisničkih imena**
- dodate atribut `rank` koji može biti između vrijednosti: `beginner`, `intermediate`, `advanced`
- atribut `email` validirajte tipom `EmailStr` iz Pydantic modula
- atribut `id` validirajte kao **pozitivan cijeli broj**
- atribut `bio` validirajte kao **string duljine minimalno 20 i maksimalno 140 znakova**
- atribut `full_name` neka bude **opcionalan** i validirajte ga kao **string duljine minimalno 3 i maksimalno 50 znakova**
- atribut `username` validirajte kao **string duljine minimalno 3 i maksimalno 20 znakova**

1.4 U glavnoj datoteci implementirajte sljedeće API rute:

- `GET /users` - vraća listu svih korisnika
- `GET /users/{id}` - vraća podatke o korisniku s traženim `id`-om (vratite odgovarajući `HTTPException` ako nije pronađen)
- `GET /users/{username}` - vraća podatke o korisniku s traženim `username`-om (vratite odgovarajući `HTTPException` ako nije pronađen)

Validirajte povratne vrijednosti za svaku od navedenih ruta kroz definirane Pydantic modele.

Dummy korisnike možete kopirati iz datoteke `users.py` (prilog uz kolokvij)

Zadatak 2 (13 bodova)

2.1 Definirajte API router za resurs `users` u zasebnoj datoteci unutar `routes` direktorija.

- uključite router u glavnu `FastAPI` aplikaciju i dodajte odgovarajući prefiks.

2.2 Implementirajte novu rutu za dodavanje korisnika.

- Ruta mora dodati novog korisnika u listu korisnika
- Potrebno je definirati dodatne `Input/Output` Pydantic modele za validaciju podataka, gdje će:
 - `Input` model sadržavati **sve attribute osim** `id`, `friends` i `rank`
 - **Definirajte novu funkciju** koja će generirati novi `id` za korisnika koristeći biblioteku `uuid` → `new_id = str(uuid.uuid4())`
 - Dodajte praznu listu prijatelja svakom novom korisniku
 - Postavite `rank` novog korisnika na `beginner`
 - `Output` model mora sadržavati sva polja koja definiraju korisnika, uključujući attribute dodane na poslužitelju
- U implementaciji rute koristite `Input` model za validaciju ulaznih podataka, a `Output` model za validaciju izlaznih podataka

2.3 Implementirajte novu rutu za dodavanje prijatelja korisniku.

- Ruta mora dodati novog prijatelja korisniku i na taj način **djelomično ažurirati resurs(e)**
- Dolazni zahtjev za ažuriranje mora sadržavati:
 - **korisničko ime korisnika kojem se dodaje prijatelj u parametru rute**
 - **korisničko ime novog prijatelja u tijelu zahtjeva**
- **Prije dodavanja novog prijatelja korisniku**, provjerite sljedeće uvjete i ispravno obradite potencijalne greške:
 - postoji li korisnik kojem se dodaje prijatelj?
 - postoji li korisnik koji se dodaje kao prijatelj?
 - je li korisnik koji se dodaje kao prijatelj već prijatelj tom korisniku?
- Iskoristite `Output` model iz zadatka **2.2** za validaciju izlaznih podataka
- **Napomena:** Ako korisniku `A` dodajete prijatelja `B`, morate dodati i korisniku `B` prijatelja `A`.

Zadatak 3 (10 bodova)

Izvan `UserProfileAPI` servisa, definirajte novi direktorij `client` u kojem ćete implementirati Python skriptu za slanje velikog broja konkurentnih HTTP zahtjeva i na taj način simulirati visoku opterećenost servisa `UserProfileAPI`.

Zahtjeve morate slati **konkurentno** i to na sljedeći način:

- Skripta mora koristiti `asyncio` i `aiohttp` biblioteke
- **Otvorite klijentsku sesiju** koristeći biblioteku `aiohttp`
- **Definirajte korutinu** `get_user_by_id` koja će slati `GET` zahtjev prema ruti

`/users/{id}` te ju pozovite unutar `main` korutine `70` puta **konkurentno**

- **Definirajte korutinu** `get_user_by_username` koja će slati `GET` zahtjev prema ruti `/users/{username}` te ju pozovite unutar `main` korutine `50` puta **konkurentno**
 - možete proizvoljno odabrati nekoliko korisničkih imena

Zadatak 4 (10 bodova)

Napravite novi `aiohttp` mikroservis `MessagingAPI` koji će **pohranjivati komunikaciju između korisnika**.

4.1 Implementirajte novu rutu `POST /message/{sender}/{receiver}` koja će omogućiti slanje poruke od pošiljatelja prema primatelju. U tijelu zahtjeva pohranite sadržaj u obliku jednog ključa, npr. `content`. U ovom zadatku ne morate raditi složenu validaciju podataka.

Primjer zahtjeva:

`POST /message/alice123/bob_dev`

```
{
  "content": "Pozdrav, kako si?"
}
```

Razgovore pohranite u obliku **liste poruka**, gdje je **svaka poruka rječnik** s atributima:

- `timestamp` - trenutak slanja poruke generirajte na poslužitelju → `(datetime.now().isoformat())`
- `content` - sadržaj poruke
- `sender` - korisničko ime pošiljatelja
- `receiver` - korisničko ime primatelja

Objekt poruke vratite u JSON odgovoru HTTP zahtjeva.

4.2 Definirajte `Dockerfile` za mikroservis `MessagingAPI` te ga pokrenite unutar Docker kontejnera.

- pohranite sve ovisnosti u `requirements.txt` datoteku te ju kopirajte u datotečni sustav kontejnera
- pokrenite mikroservis na kontejnerskom portu `1202` i izložite ga prema domaćinu
- testirajte rad mikroservisa uz pomoć HTTP klijenta po izboru

Dodajte u komentare mikroservisa terminal naredbe koje ste koristili za **izgradnju predloška** (`Dockerfile`) i **pokretanje kontejnera**.

Zadatak 5 (13 bodova)

5.1 Implementirajte novu rutu `POST /simulate` u mikroservisu `UserProfileAPI` u kojem ćete simulirati jednostavnu komunikaciju između 2 korisnika.

Odaberite način kako ćete poslati podatke na rutu, primjerice u tijelu zahtjeva:

```
{
  "user_1" : "alice123",
  "user_2" : "bob_dev",
}
```

Otvorite `aiohttp` klijentsku sesiju i **pošaljite 2 sekvencijalna HTTP zahtjeva prema mikroservisu** `MessagingAPI`, a za `content` uzmite biografije korisnika iz servisa `UserProfileAPI`.

Primjer simulacije:

- `alice123` šalje poruku `bob_dev` sa sadržajem `A passionate software developer.`
- `bob_dev` šalje poruku `alice123` sa sadržajem `Loves coding and open-source projects.`

5.2 Definirajte `Dockerfile` za mikroservis `UserProfileAPI` te ga pokrenite unutar Docker kontejnera.

- pohranite sve ovisnosti u `requirements.txt` datoteku te ju kopirajte u datotečni sustav kontejnera
- pokrenite mikroservis na kontejnerskom portu `8000` i izložite ga prema domaćinu
- testirajte rad mikroservisa pokrenutog u kontejneru kroz Swagger sučelje

Dodajte u komentare mikroservisa terminal naredbe koje ste koristili za **izgradnju predloška** (`Dockerfile`) i **pokretanje kontejnera**.

5.3 Koristeći `Docker Compose` alat povežite mikroservise `UserProfileAPI` i `MessagingAPI` preko zajedničke mreže te ih pokrenite skupa, kao cjelinu.

- izmijenite implementaciju rute `POST /simulate` u mikroservisu `UserProfileAPI` tako da koristi **ispravnu domenu** za slanje zahtjeva prema mikroservisu `MessagingAPI`.

Ukupni raspodijeljeni sustav (gdje se nalazi Compose konfiguracijska datoteka) mora biti predan u direktoriju `rs-final`.